K.J. Somaiya College of Engineering
Department of Computer Engineering

Advanced Cloud Computing (ACC) — Continuous Assessment

ML-Based Memory Usage Prediction and Intelligent Cloud Resource Allocation

Design, simulation and evaluation of a forecast-driven auto-scaling resource manager

Names: Piyush Chaudhary 16010123234
Aswin Nambiar 16010123086
Programme: B.Tech Computer Engineering
Source code: https://github.com/Piyushchy/acc-lab-ca

October 2026

Abstract

Cloud services are commonly provisioned either for their peak load, which wastes money and energy, or reactively, which adds capacity only after demand has risen and therefore risks running out of memory while new instances boot. This report designs, implements and evaluates an ML-based resource manager that forecasts the memory demand of a service 15 minutes ahead and sizes a pool of virtual machines accordingly. A leakage-free set of 32 features is built from memory, request-rate and CPU history plus calendar information. Seven forecasters are compared on a chronological hold-out; an MLP reaches a mean absolute error of 252.6 MB (1.01 % MAPE), about half the error of the "no change" baseline. The forecast, in the form of a 95th-percentile upper bound, drives a discrete-time auto-scaling simulator that is compared with static peak provisioning, reactive target tracking and an oracle on prediction accuracy, utilisation, latency, cost and energy. Dynamic scaling raises average utilisation from 50 % to about 78 % and reduces simulated cost by about 36 % and energy by about 21 %. The report then studies what happens when the prediction is wrong: systematic under-prediction is dangerous (28.9 % of time steps with exhausted memory at -20 % bias), over-prediction merely costs money, and for sudden unforeseen surges the instance launch delay, not the forecaster, is the binding constraint. A drift-detecting hybrid policy removes the bias failure at a cost premium. All results use a synthetic workload and an assumed cost/power model; this limitation is stated prominently and discussed throughout.

Contents

1 Introduction and Problem Statement **2**

2 Input Data and Features **2**

3 Algorithm Selection and Prediction Horizon **4**

4 Resource Allocation and Auto-Scaling Mechanism **8**

5 Evaluation Methodology **9**

6 Results **9**

7 What Happens When the Prediction Is Wrong? **12**

8 Cloud Demonstration **16**

9 Limitations **16**

10 Conclusion **16**

1 Introduction and Problem Statement

Elasticity is a defining property of cloud computing: resources can be acquired and released on demand [**?**]. Exploiting it requires deciding *how many* resources to hold at each moment. Peak-sized (static) provisioning guarantees capacity but leaves most of it idle; reactive auto-scalers, such as the target-tracking policies of commercial platforms [**?**], respond to *current* utilisation, so capacity arrives one launch-and-warm-up delay after the demand it was meant for [**?**]. Predictive approaches address this by forecasting the workload and acquiring capacity in advance [**? ? ?**].

Problem. Design an ML-based cloud resource manager that predicts workload and allocates resources. The assignment asks us to

(i) identify the input data and features,

(ii) choose the ML algorithm and prediction horizon,

(iii) design the resource-allocation / auto-scaling mechanism,

(iv) evaluate prediction accuracy, utilisation, latency, cost and energy, and

(v) discuss what happens when the prediction is wrong.

Sections 2–7 address these items in order; Section 8 describes the cloud demonstration.

Scope and honesty statement. The default workload is *synthetic* and the cluster is a *discrete-time simulator* with assumed price and power figures. No headline number was measured on a live cloud account. The code can ingest any real trace and includes scripts to collect metrics on a cloud VM and to push recommendations to an AWS Auto Scaling Group, but the latter has only been run in dry-run mode. Consequently, absolute values (dollars, kWh, milliseconds) illustrate *relative* differences between policies and should not be read as predictions for a real deployment.

2 Input Data and Features

2.1 Workload data

The trace covers 60 days at 5-minute resolution (17,280 rows) with three columns: request rate (req/s), CPU utilisation (%) and aggregate service memory demand (MB, range 13–40 GB). The generator (`src/data_gen.py`) combines diurnal and weekly seasonality, 12 % long-term growth, AR(1) noise and about one traffic burst per day; CPU is proportional to the request rate; memory is a baseline plus an exponentially smoothed response to the request rate plus a slow leak that resets at simulated deployments. Figure 1 shows the series.

The data are split *chronologically*: 60 % train (≈35 days), 20 % validation and 20 % test (≈11.8 days), without shuffling, so every model is evaluated on the future relative to its training data [**?**].

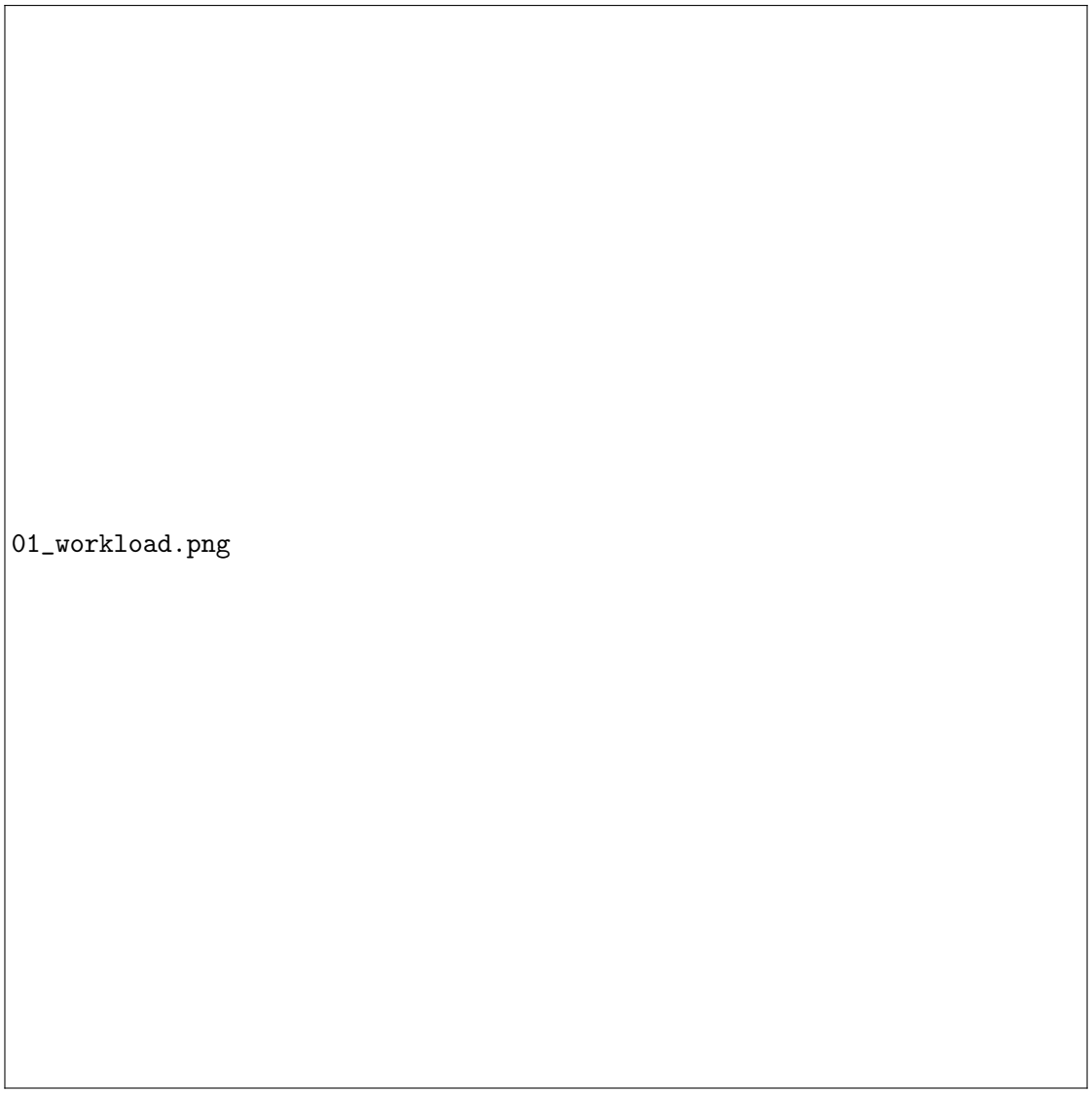


Figure 1: Synthetic workload: request rate, CPU and memory demand, with the train/validation/test split.

2.2 Features

All 32 features at time t use only data available up to t; a unit test (`tests/test_pipeline.py`) verifies the absence of look-ahead leakage.

- **Memory history:** current level and differences over 5 min to 24 h (lags of 1, 2, 3, 6, 12, 36, 72, 144 and 288 steps).
- **Rolling statistics:** deviation from the mean, standard deviation and deviation from the maximum over 30 min, 1 h and 3 h windows.
- **Seasonal signal:** the memory change over the same window *yesterday*.
- **Request and CPU signals:** request level, differences, exponential moving average, 3 h mean, CPU level and 15-min CPU change. Request rate is a leading indicator of memory.
- **Calendar features** of the *target* time: $\sin / \cos$ of hour of day and day of week, and a weekend flag.

2.3 Target

The target is the memory *change* over the horizon h, $y_t = m_{t+h} - m_t$; the forecast is $\hat{m}_{t+h} = m_t + \hat{y}_t$. Predicting the change rather than the level prevents tree-based models from having to extrapolate beyond the memory range seen in training, which matters because the growth trend pushes the test period above the training range.

3 Algorithm Selection and Prediction Horizon

3.1 Candidate models

Three baselines (persistence, 1-hour moving average, seasonal-naive using yesterday's change) and four learned regressors were compared: Ridge regression [**?**], Random Forest [**?**], gradient boosting (scikit-learn's histogram implementation) [**?**] and a two-hidden-layer MLP (64–32 units) [**?**], all implemented with scikit-learn [**?**]. Models are fit on the training set, the best learned model is selected by *validation* MAE, all models are then refit on train+validation, and the test set is scored once.

Table 1: Forecast accuracy on the held-out test period (15-minute horizon).

Model	MAE (MB)	RMSE (MB)	MAPE	Skill vs. persistence
Persistence	497.0	648.1	2.01 %	0 %
Moving average (1 h)	1249.5	1575.6	5.05 %	−151 %
Seasonal naive (24 h)	436.8	597.5	1.75 %	12.0 %
Ridge regression	256.0	367.4	1.02 %	48.5 %
Random Forest	259.5	372.0	1.03 %	47.8 %
Gradient Boosting	260.5	371.8	1.03 %	47.6 %
MLP (64–32)	**252.6**	**361.9**	**1.01 %**	**49.2 %**

The four learned models lie within about 3 % of each other and a linear model is competitive. This is expected because the synthetic memory is generated by nearly linear dynamics; Table 1 therefore says little about which model family would win on real traces. The MLP was selected by the validation rule. Figures 2 and 3 show the comparison and the forecast against the actual series.

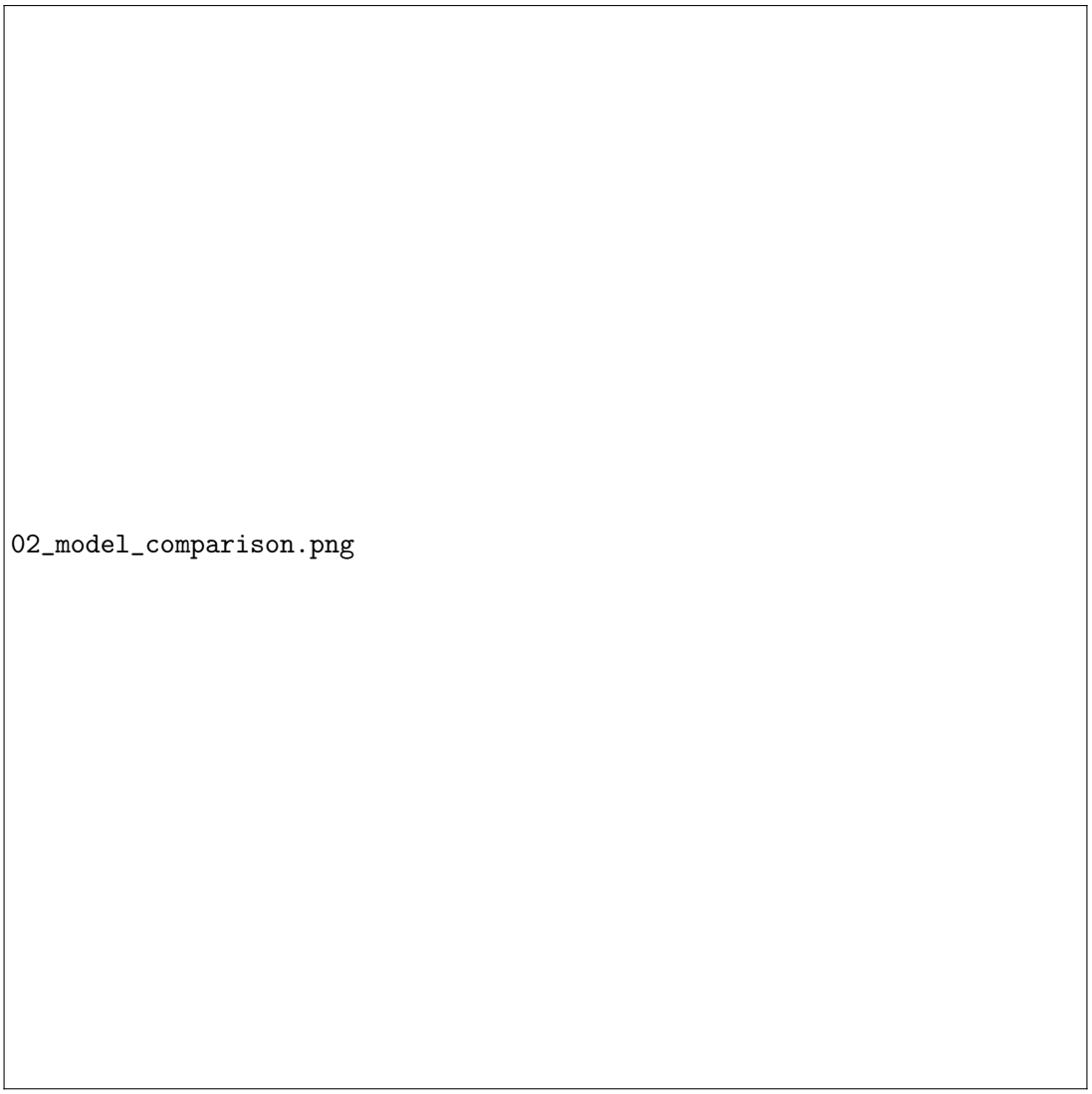


Figure 2: Model comparison on the test set.

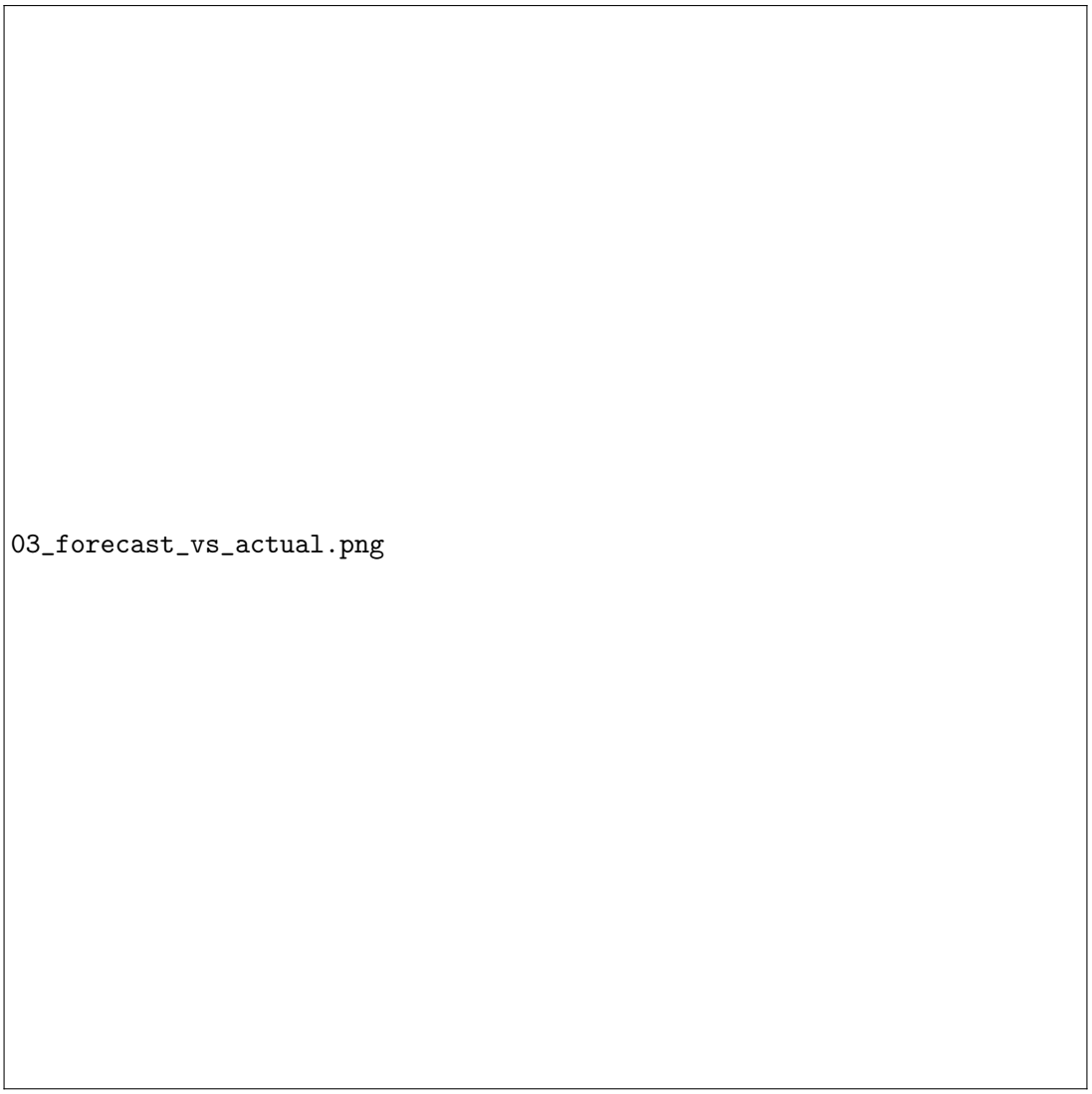


Figure 3: Forecast versus actual memory over part of the test period.

3.2 Upper-bound model

Capacity must cover the demand that actually occurs, not its mean, so a gradient-boosting *quantile* model ($q = 0.95$) [**?**] supplies an upper bound. On the test set its empirical coverage is 90.8 % rather than the nominal 95 %: it is somewhat over-confident, plausibly because of the upward trend between training and test periods. A conformal calibration step would be a natural improvement. Permutation importance (Figure 4) ranks the target-time hour, short-term request-rate changes and the weekend flag highest.

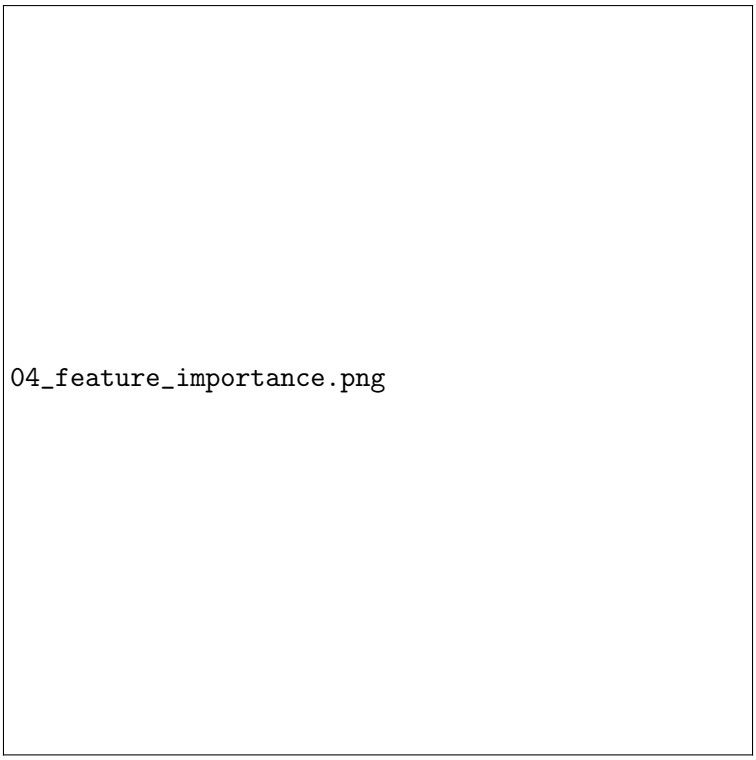


Figure 4: Permutation feature importance.

3.3 Prediction horizon

The default horizon is **15 minutes (3 steps)**, equal to the assumed instance launch and warm-up time: a decision made now only matters for the moment the new capacity becomes available. Table 2 and Figure 5 sweep the horizon.

Table 2: Effect of the prediction horizon (hybrid policy).

Horizon	5 min	10 min	15 min	30 min	60 min
Forecast MAE (MB)	97	176	261	489	810
Persistence MAE (MB)	179	340	497	942	1756
Cost (USD)	91.7	92.2	92.7	93.5	94.7
Steps with memory exhausted	0 %	0 %	0 %	0 %	0 %

Error grows roughly linearly with the horizon and cost rises slightly because more headroom is needed. On this smooth series, horizons shorter than the launch delay did not cause violations, so the 15-minute choice rests on the launch-delay argument rather than on a measured penalty.

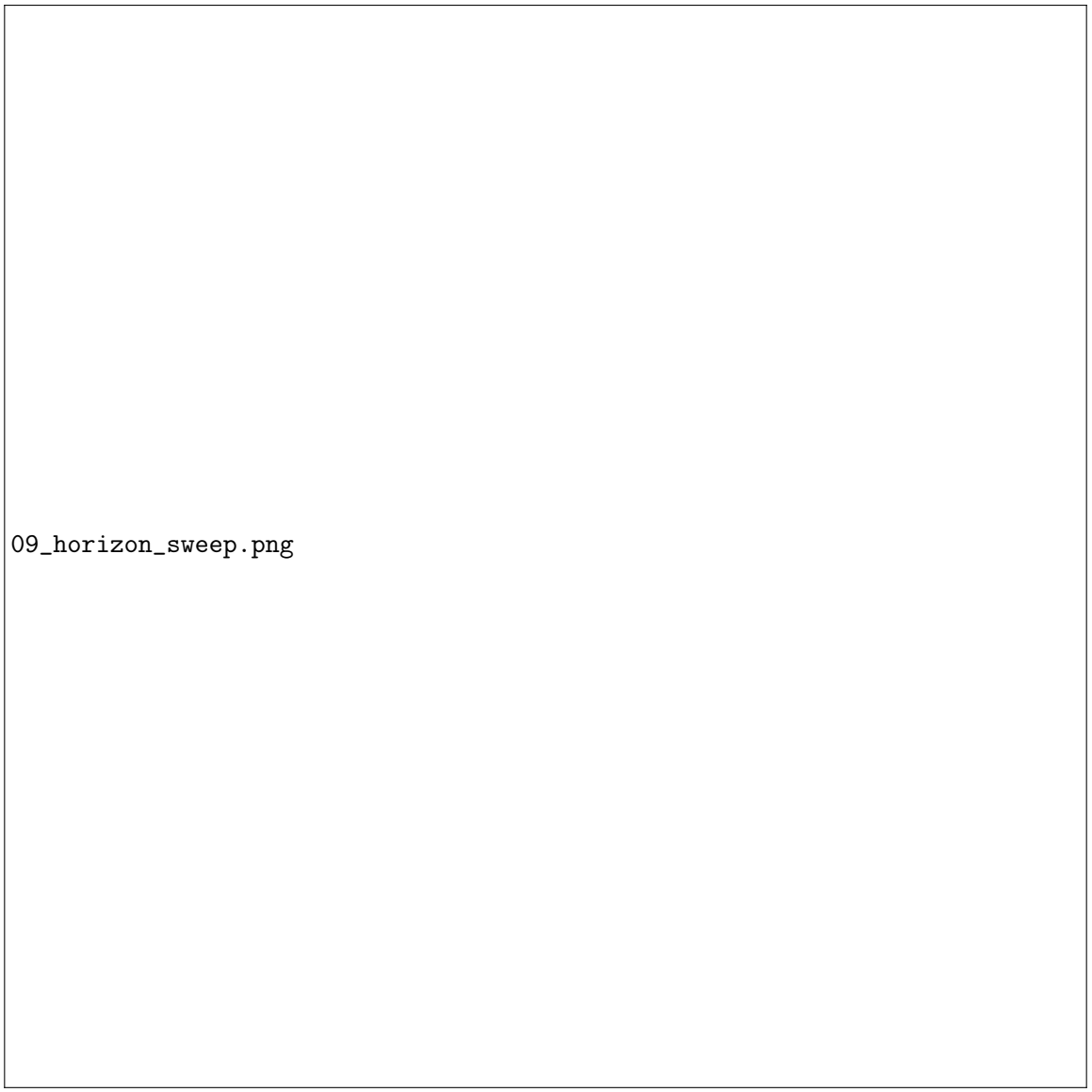


Figure 5: Horizon sweep: forecast error and policy cost.

4 Resource Allocation and Auto-Scaling Mechanism

The simulated service runs on instances of 4 GB memory with a 15-minute launch and warm-up delay and a pool of 2–80 instances. At each 5-minute step the controller applies:

1. **Sizing rule.**
$$N_t = \left\lceil \frac{\hat{m}_{t+h}^{(0.95)}}{M_{\text{inst}} \cdot u^\star} \right\rceil, \qquad M_{\text{inst}} = 4096\,\text{MB},\ u^\star = 0.85, \tag{1}$$
where $\hat{m}_{t+h}^{(0.95)}$ is the q95 forecast (the mean forecast is used for the "predictive-mean" policy).
2. **Immediate scale-out.** The request is issued at once, but capacity arrives only after the launch delay; billing and idle power start at launch.
3. **Stabilised scale-in.** The pool shrinks only to the highest desired size seen in the last 30 minutes (anti-flapping), identically for all dynamic policies.

4. **Safety nets (hybrid policy).** An emergency guard acts on current utilisation, and a drift fallback replaces the model when realised error is high (Section 7).

Policies compared. (1) *Static*: training peak plus 20 %, i.e. 12 instances; (2) *Reactive*: the same sizing rule fed with *current* demand (target tracking); (3) *Predictive (mean)*; (4) *Predictive (q95)*; (5) *Hybrid*: q95 plus safety nets; (6) *Oracle*, which knows the true demand at $t+h$ and bounds what any forecaster can achieve.

5 Evaluation Methodology

- **Prediction accuracy:** MAE, RMSE, MAPE, R^2, skill versus persistence, under-prediction rate and q95 coverage.
- **Utilisation:** mean of $\min(\text{demand}/\text{capacity}, 1)$.
- **Reliability:** percentage of steps in which demand exceeds capacity (memory exhausted), number of such episodes and shortfall in GB-hours.
- **Latency (proxy):** $40\,\text{ms}/(1-u)$, capped at $20\times$ below saturation and set to 2000 ms when memory is exhausted; mean, p95 and p99 are reported.
- **Cost:** billed instance-hours at \$0.0416 per instance-hour.
- **Energy:** per-instance power rising linearly from 45 W idle to 110 W at full utilisation [**?**]; booting instances draw idle power.

Prices, power numbers and the latency model are *assumptions*. The latency proxy is a queueing-style heuristic, not a measurement.

6 Results

Table 3: Policy comparison on the test period (85 % sizing target).

Policy	Util.	Mem. exhausted	p99 (ms)	Cost ($)	Energy (kWh)	Avg inst.
Static (peak-sized)	50.3 %	0 %	194	141.36	264.0	12.0
Reactive	78.6 %	0.03 %	333	90.95	209.5	7.7
Predictive (mean)	78.5 %	0 %	279	90.95	209.5	7.7
Predictive (q95)	77.0 %	0 %	255	92.71	211.4	7.8
Hybrid	77.0 %	0 %	254	92.72	211.4	7.8
Oracle	78.5 %	0 %	263	90.98	209.5	7.7

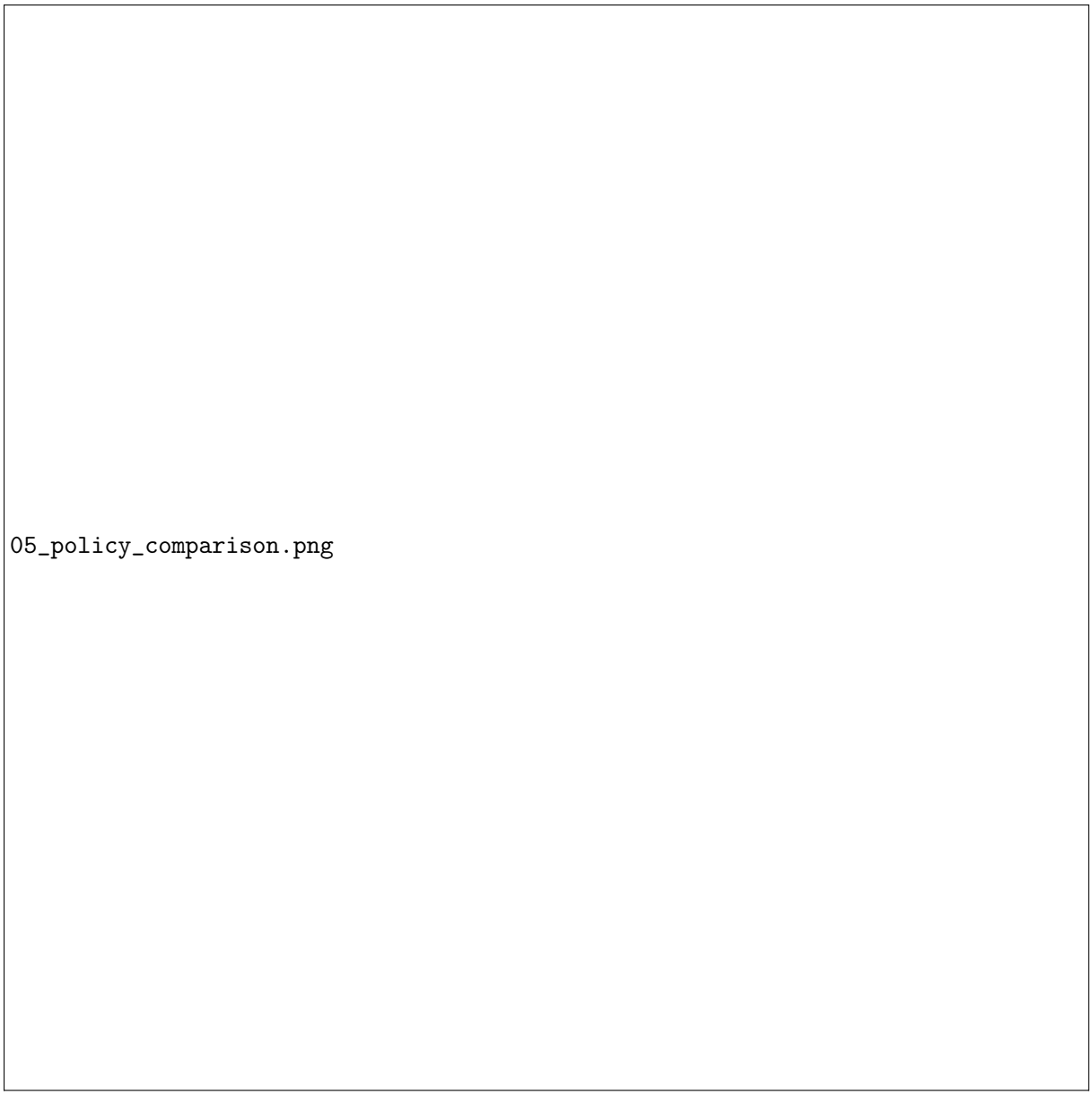


Figure 6: Utilisation, violations, latency, cost and energy by policy.

Every dynamic policy reduces cost by about 36 % and energy by about 21 % relative to static peak sizing, at the price of higher latency because the cluster runs hotter, which the latency proxy penalises. At the default 85 % target the differences between reactive and ML policies are small: slow diurnal ramps fit within 15 % headroom even with a 15-minute delay. The ML policies avoid the single short reactive violation and lower p99 latency, at equal cost for the mean forecast and +2 % for the q95 headroom. Figure 7 shows capacity tracking demand.

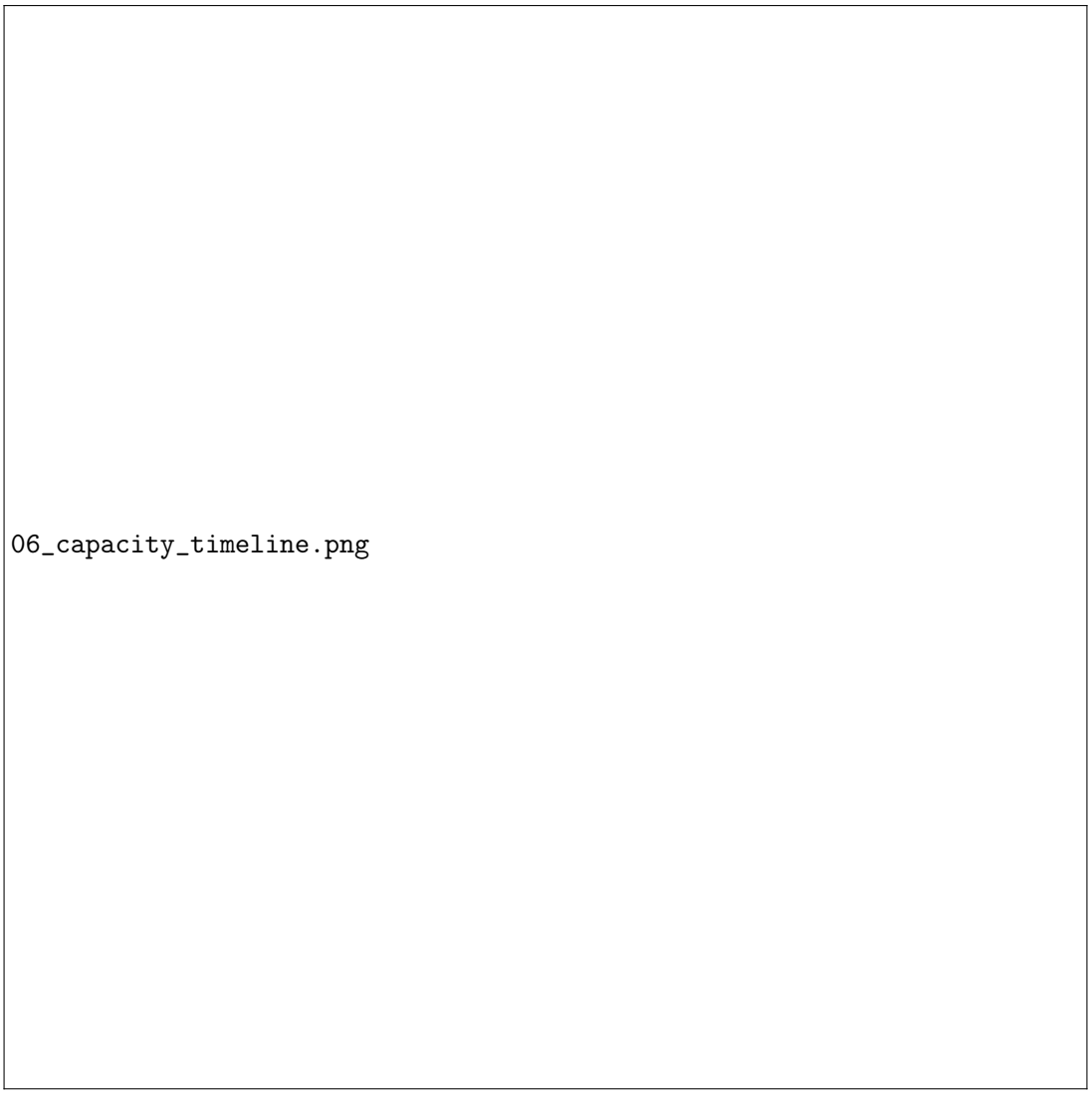


Figure 7: Capacity versus demand over time for the compared policies.

6.1 Where prediction matters: tighter packing

Because the default target hides differences, the sizing target $u^\star$ was swept (Table 4, Figure 8). The forecast allows tighter packing with fewer violations at nearly the same cost; the benefit is real in this simulation but small in absolute terms (fractions of a percent of steps). The sweep was added after a first run at a 75 % target showed zero violations for all dynamic policies; all points are reported, not only favourable ones.

Table 4: Sizing-target sweep, normal workload.

	Reactive		Predictive (mean)	
Target $u^\star$	Violations	Cost ($)	Violations	Cost ($)
70 %	0 %	109.1	0 %	109.1
85 %	0.03 %	91.0	0 %	91.0
90 %	0.06 %	86.0	0 %	86.1
95 %	0.24 %	82.1	0.03 %	82.3

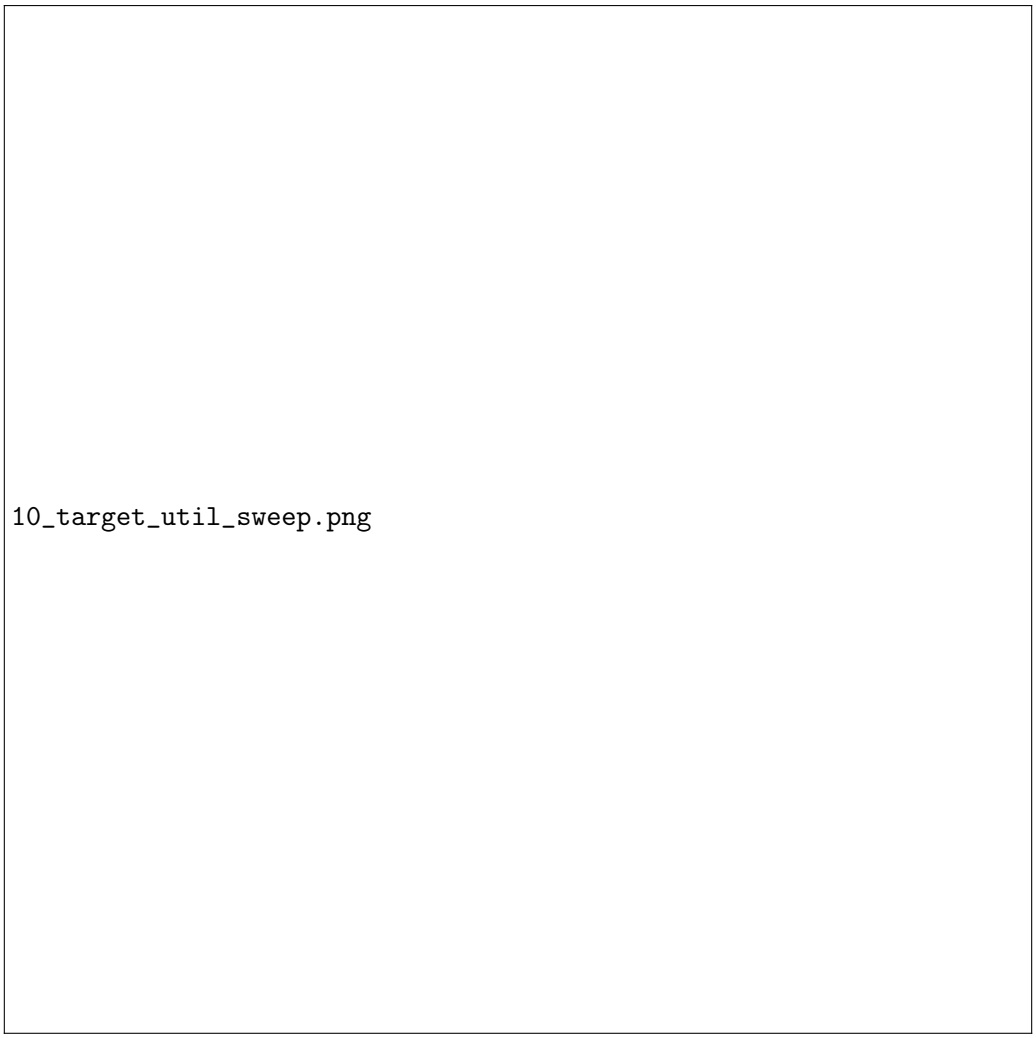


Figure 8: Sizing-target sweep.

7 What Happens When the Prediction Is Wrong?

Four experiments probe failure behaviour (Figures 9, 10, 11).

7.1 Systematic bias

The forecast was multiplied by a constant factor (Table 5). A 10 % under-forecast is still safe because of sizing headroom, but at -20 % the predictive-only policy leaves memory exhausted in 28.9 % of steps (121 episodes) and at -40 % in essentially all steps. Over-prediction is safe but costly: $+40$ % raises cost from \$91 to \$125 (+37 %) with no violations. **Under-prediction is the dangerous direction; over-prediction only wastes money**, an asymmetry that favours an upper quantile over the mean.

Table 5: Effect of a multiplicative forecast bias (factor applied to the forecast).

	Predictive only		Hybrid (drift fallback)	
Factor	Mem. exhausted	Cost ($)	Mem. exhausted	Cost ($)
0.6	99.97 %	56.8	0.09 %	116.4
0.7	88.08 %	65.5	0.09 %	116.4
0.8	28.87 %	74.0	0 %	116.4
0.9	0.03 %	82.8	0 %	85.9
1.0	0 %	91.0	0 %	92.7
1.2	0 %	107.9	0 %	116.4
1.4	0 %	125.0	0 %	127.2

7.2 Random noise

Multiplicative noise of up to 20 % on the forecast causes no violations, but cost rises from $91 to $111 as the policy over-provisions on noisy up-swings; at 30 % noise the predictive-only policy reaches 0.18 % violations while the hybrid stays at 0 %.

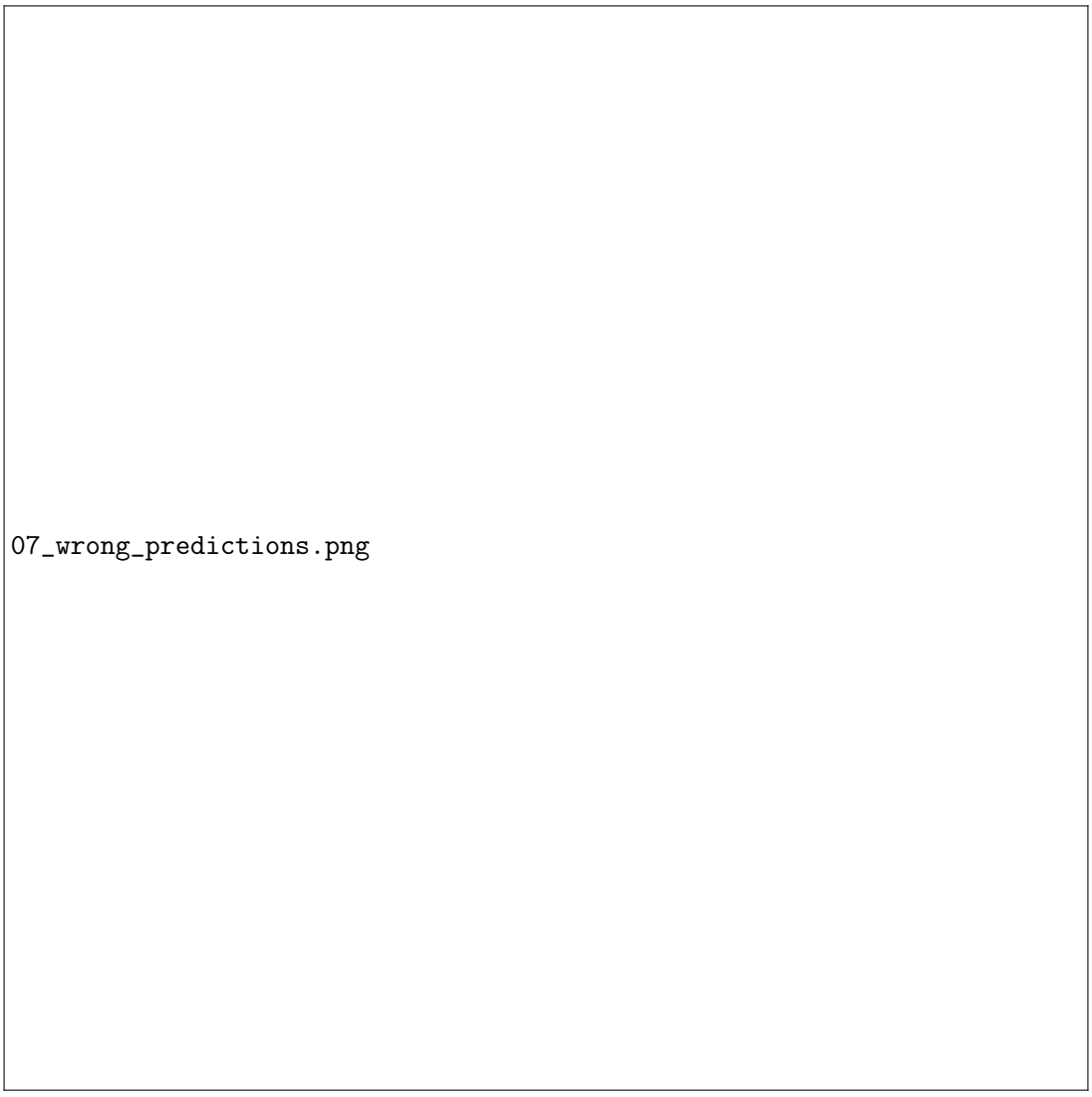


Figure 9: Wrong-prediction experiments: bias and noise.

7.3 Unforeseen surges

A +40 % memory surge for 90 minutes and a +60 % surge for 60 minutes were injected into the test period without changing the request rate, so no feature can foresee them (Table 6, Figure 10). Memory is exhausted in 0.18 % of steps for the reactive policy and 0.15 % for predictive and hybrid; the oracle has none. **The safety nets did not help here**: the emergency guard and the drift fallback fired (in 0.2 % and 0.6 % of steps), but new capacity still needs 15 minutes to arrive. The launch-delay sweep (Table 7, Figure 11) confirms that the delay is the binding constraint: with identical forecasts, violations grow from 0.03 % (5-minute launch) to 0.41 % (30 minutes), while the oracle goes from 0 % to 0.18 %.

Table 6: Surge scenario (unforeseeable memory spikes).

Policy	Mem. exhausted	Episodes	p99 (ms)	Cost ($)
Static	0 %	0	195	141.36
Reactive	0.18 %	3	341	91.42
Predictive (mean)	0.15 %	2	283	91.38
Predictive (q95)	0.15 %	2	260	93.18
Hybrid	0.15 %	2	259	93.49
Oracle	0 %	0	263	91.44

08_shock_scenario.png

Figure 10: Surge scenario: demand and capacity.

Table 7: Violations (% of steps) versus instance launch delay in the surge scenario.

Launch delay (min)	Reactive	Predictive (mean)	Hybrid	Oracle
5	0.03	0.03	0.03	0
10	0.09	0.09	0.09	0
15	0.18	0.15	0.15	0
30	0.44	0.41	0.41	0.18



Figure 11: Launch-delay sweep in the surge scenario.

7.4 Drift fallback

The hybrid compares each forecast with the value that actually occurred h steps later. If the mean absolute percentage error over the last hour exceeds 12 %, it stops trusting the model and sizes reactively with 30 % headroom. With a -20 % biased forecast the hybrid keeps violations at 0 % (versus 28.9 % for predictive-only) at \$116 versus \$93 for the unbiased hybrid: insurance paid for in cost. In the unbiased run it never triggered, so it produced no false alarms on this data.

7.5 Mitigations

1. Forecast an upper quantile rather than the mean (implemented).
2. Monitor realised forecast error, fall back to reactive sizing, and retrain when error stays high (fallback implemented; automated retraining is the intended operational response but is not implemented).
3. Emergency guard on current utilisation (implemented).
4. Minimum-capacity floor and stabilised scale-in (implemented).
5. Shorter effective launch time through pre-warmed pools, smaller images or containers, or standing headroom. Not implemented, but indicated by the surge experiments: no policy can react faster than capacity can be created.
6. Application-level protection such as memory limits and request shedding.

8 Cloud Demonstration

The repository supports the demonstration on any cloud platform (instructions in `README.md` and `docs/DEMO_SCRIPT.md`):

1. create a small VM (AWS EC2, Google Compute Engine or Azure VM), clone the repository and install `requirements-cloud.txt`;
2. run `python main.py` to reproduce the results, or use the provided `Dockerfile`;
3. generate a real memory workload with `cloud/load_generator.py`, record it with `cloud/collect_metrics.py`, and retrain with `python main.py -data data/collected_vm.csv`;
4. optionally apply the recommendation to an Auto Scaling Group with `cloud/aws_apply.py` (dry run by default) and publish the forecast as a CloudWatch custom metric.

Status. Steps 1–3 were exercised locally, including the `-data` path; `aws_apply.py` was run only in dry-run mode and has not been tested against a live AWS account. Seven automated tests pass (`pytest`). The screencast follows the script in `docs/DEMO_SCRIPT.md`.

9 Limitations

- The data are synthetic, with a built-in "requests lead memory" relationship and near-linear dynamics. Real traces are less predictable, real accuracy will be lower and the model ranking may change.
- Costs, power and latency use assumed parameters; the simulator ignores spot pricing, billing granularity, multi-dimensional resources (CPU, network), bin-packing and application behaviour.
- One random seed (42) and one test window; no confidence intervals. Policy differences at the default operating point are small.
- The q95 model is under-covered (90.8 %).

10 Conclusion

Short-horizon memory forecasting (about 1 % MAPE here, roughly half the error of "no change") combined with an upper-quantile sizing rule lets a cluster run at about 78 % utilisation instead of 50 %,

cutting simulated cost by about a third and energy by about a fifth relative to peak provisioning, with violation rates at or below those of a reactive autoscaler. The experiments also expose the limits: when forecasts are biased low, a predictive-only policy fails badly and needs the drift fallback, and when the surprise is a sudden surge, the instance launch delay rather than the forecasting method limits what any policy can achieve. Future work includes validation on real production traces, conformal calibration of the upper bound, automated retraining, multi-resource forecasting and live-cloud measurements.

References